

Built-in Delegate Types in .NET

To save you from declaring your own delegate types every time, .NET provides three built-in generic delegates in the System namespace:

Delegate Type Description		Return Type
Action	Encapsulates a method that does not return a value	void
Func	Encapsulates a method that returns a value	Any type
Predicate	Encapsulates a method that returns a bool (for testing a condition)	bool

1. Action<T> — For Methods That Return Nothing

Action delegates are used when you want to execute code but don't expect a return value.

Key Points:

- Action can have 0 to 16 input parameters.
 - Always returns void.
 - Commonly used for callbacks or event handlers.
-

2. Func<T, TResult> — For Methods That Return a Value

Func delegates are used when you need to return a value.

Key Points:

- The last type parameter is always the return type.
Example: `Func<int, string>` → takes an int, returns a string.
 - Func can have 0 to 16 input parameters.
 - Perfect for transformations, calculations, or LINQ expressions.
-

3. Predicate<T> — For Boolean Conditions

A Predicate is a specialized delegate that takes one input and returns a bool.

Key Points:

- Used mainly for filtering or testing conditions.
- Common in `List<T>` methods like:
- `List<int> numbers = new() { 1, 2, 3, 4, 5 };`

- `var evenNumbers = numbers.FindAll(isEven);`

Quick Comparison

Delegate	Parameters	Return Type	Example
Action<T1, T2>	Up to 16	void	Action<int, string> log = (id, msg) => ...
Func<T1, T2, TResult>	Up to 16 (last = return type)	Any	Func<int, int, int> add = (x, y) => x + y;
Predicate<T>	1	bool	Predicate<int> isEven = x => x % 2 == 0;

Why Built-in Delegates Matter

Before these existed, developers had to manually declare custom delegates for every scenario:

```
public delegate int MyDelegate(int x, int y);
```

Now, you just use:

```
Func<int, int, int> myFunc = (x, y) => x + y;
```

Cleaner, reusable, and integrates seamlessly with LINQ, async, and event-driven programming.

Summary

Concept Description

Delegate A type-safe method reference.

Action Runs a method with parameters but returns nothing.

Func Runs a method and returns a value.

Predicate Runs a method that checks a condition and returns a bool.

Use Case Callbacks, event handlers, LINQ queries, async workflows.